

複数デポ時間枠付き配送計画問題の混合整数計画による求解と

Timefold メタヒューリスティクス解との比較

Mikio Kubo

2026 年 10 月 7 日

概要

制約ソルバー Timefold のクイックスタート (vehicle-routing) で扱われている複数デポ時間枠付き配送計画問題 (MDVRPTW) を、2 添字アーク定式化に車両割当変数を加えた混合整数計画として定式化し、AMPL と Python API (amplpy) で実装した。フィラデルフィア近郊の訪問先 55 件・車両 6 台のインスタンスで、Timefold が出力した解 (総走行時間 82,660 秒) と比較した。Gurobi は 600 秒の計算で総走行時間 82,210 秒 (−0.54%) の解を得た。同時に得た下界 77,652 秒から、Timefold の解の最適性ギャップは高々 6.1% であることが分かった。LP 緩和に部分巡回除去制約を加えると下界は 66,625 秒から 74,453 秒へ改善するが、時間枠に由来する big-M 制約の弱さのため、最適性の証明には至らなかった。

1 はじめに

Timefold Solver は、局所探索などのメタヒューリスティクスを用いた制約ソルバーである。公式のクイックスタート^{*1} には、車両ごとに異なるデポ (車庫)・容量をもつ時間枠付き配送計画問題の例題が含まれている。メタヒューリスティクスは短時間で良い解を出すか、その解が最適値からどれだけ離れているかは分からない。

本稿では同じ問題を混合整数計画 (MIP) として定式化し、次の 3 点を調べる。

1. Timefold の解を MIP ソルバーで改善できるか。
2. MIP ソルバーの下界から、Timefold の解の品質 (最適性ギャップ) をどこまで保証できるか。
3. 定式化の工夫 (アークの前処理、部分巡回除去制約、インジケータ制約) が計算にどう効くか。

2 問題

2.1 Timefold の問題定義

Timefold のクイックスタートでは、次の制約とスコアで解を評価する。

- **ハード制約**: 車両容量 (vehicleCapacity) と、サービス終了時刻が最大終了時刻を超えな

^{*1} <https://github.com/TimefoldAI/timefold-quickstarts/tree/stable/use-cases/vehicle-routing>

いこと (serviceFinishedAfterMaxEndTime).

- ソフト制約：総走行時間の最小化 (minimizeTravelTime). デポへの帰着も含む.
- すべての訪問先を, どれか 1 台の車両に割り当てる.

各車両は自分のデポを出発時刻に出発し, 訪問先を順に回って同じデポに戻る. 訪問先に最早開始時刻より早く着いた場合は待つ.

2.2 インスタンス

Timefold の Web UI から保存した `フィラデルフィア.json` を用いる. 表 1 に概要を示す. このファイルには問題データに加えて Timefold が求めた解 (各車両の訪問順) も含まれており, スコアは 0hard/0medium/-82660soft, つまり総走行時間 82,660 秒である.

表 1 インスタンス (フィラデルフィア) の概要

項目	値
訪問先数	55 (午前枠 28 件, 午後枠 27 件)
車両数 (=デポ数)	6
車両容量	24, 28, 27, 15, 27, 28 (合計 149)
総需要	81
出発時刻	全車両 7:30
時間枠	午前 8:00–12:00, 午後 13:00–18:00
サービス時間	10, 20, 30, 40 分

2.3 走行時間

Timefold の `HaversineDrivingTimeCalculator` は, 緯度経度から大円距離を求め, 平均時速 50 km で走行時間に換算する. 本稿では次の式を用いた.

$$d_{pq} = \left\lfloor 2R \arcsin \sqrt{h_{pq}} + 0.5 \right\rfloor \text{ [m]}, \quad t_{pq} = \left\lfloor \frac{3.6 d_{pq}}{50} + 0.5 \right\rfloor \text{ [s]}, \quad (1)$$

ここで $R = 6,371,000$ m, $h_{pq} = \sin^2 \frac{\phi_q - \phi_p}{2} + \cos \phi_p \cos \phi_q \sin^2 \frac{\lambda_q - \lambda_p}{2}$ (ϕ は緯度, λ は経度) である. 床関数 $\lfloor x + 0.5 \rfloor$ は Java の `Math.round` に合わせた四捨五入である. 距離をメートルに丸めてから秒に丸めるこの式は, JSON に記録された 55 区間の走行時間と完全に一致した. 距離を丸めない式や切り捨てを使う式では 1–3 区間で 1 秒ずれる.

3 定式化

3.1 記号

- V : 訪問先の集合, K : 車両の集合. 車両 k のデポを o_k と書く.
- q_i : 訪問先 i の需要, s_i : サービス時間, $[e_i, l_i]$: 最早開始時刻と最大終了時刻 (l_i はサービス終了の期限).

- Q_k : 車両 k の容量, τ_k : 出発時刻.
- t_{ij} : 訪問先間, t_{kj} : デポ o_k から訪問先 j , t_{ik} : 訪問先 i からデポ o_k への走行時間.

時刻は問題の開始時刻 (7:30) からの秒で表す.

3.2 アークの前処理

時間枠を守れないアークを事前に除く.

$$A = \{(i, j) \in V \times V : i \neq j, e_i + s_i + t_{ij} + s_j \leq l_j\}, \quad (2)$$

$$S = \{(k, j) \in K \times V : \tau_k + t_{kj} + s_j \leq l_j\}. \quad (3)$$

たとえば午後枠の訪問先から午前枠の訪問先へのアークは, 午後の最早開始時刻 13:00 が午前の期限 12:00 より遅いので除かれる. この前処理で訪問先間のアークは 2,970 本から 2,210 本に減る. デポからのアーク S は 330 本すべて残る.

3.3 変数

- $x_{ij} \in \{0, 1\}$ $((i, j) \in A)$: 訪問先 i の次に j を訪問するとき 1.
- $y_{kj} \in \{0, 1\}$ $((k, j) \in S)$: 車両 k の最初の訪問先が j のとき 1.
- $z_{ik} \in \{0, 1\}$ $(i \in V, k \in K)$: 訪問先 i の後に車両 k がデポへ戻るとき 1.
- $a_{ik} \in \{0, 1\}$ $(i \in V, k \in K)$: 訪問先 i を車両 k が担当するとき 1.
- $T_i \in [e_i, l_i - s_i]$: 訪問先 i のサービス開始時刻.

アーク変数 x は車両添字をもたない (2 添字定式化). 車両ごとにデポ・容量が異なるため, 誰がどの訪問先を担当するかを割当変数 a でルートに沿って伝える. 車両添字付きのアーク x_{ijk} を使う 3 添字定式化に比べ, 2 進変数が約 $|K|$ 分の 1 で済む.

3.4 モデル

$$\min \sum_{(i,j) \in A} t_{ij} x_{ij} + \sum_{(k,j) \in S} t_{kj} y_{kj} + \sum_{i \in V} \sum_{k \in K} t_{ik} z_{ik} \quad (4)$$

$$\text{s.t.} \quad \sum_{(i,j) \in A} x_{ij} + \sum_{(k,j) \in S} y_{kj} = 1 \quad j \in V \quad (5)$$

$$\sum_{(i,j) \in A} x_{ij} + \sum_{k \in K} z_{ik} = 1 \quad i \in V \quad (6)$$

$$\sum_{(k,j) \in S} y_{kj} \leq 1 \quad k \in K \quad (7)$$

$$\sum_{i \in V} z_{ik} = \sum_{(k,j) \in S} y_{kj} \quad k \in K \quad (8)$$

$$\sum_{k \in K} a_{ik} = 1 \quad i \in V \quad (9)$$

$$y_{kj} \leq a_{jk} \quad (k,j) \in S \quad (10)$$

$$z_{ik} \leq a_{ik} \quad i \in V, k \in K \quad (11)$$

$$x_{ij} + a_{ik} - a_{jk} \leq 1 \quad (i,j) \in A, k \in K \quad (12)$$

$$\sum_{i \in V} q_i a_{ik} \leq Q_k \quad k \in K \quad (13)$$

$$T_j \geq \tau_k + t_{kj} - M_{kj}(1 - y_{kj}) \quad (k,j) \in S \quad (14)$$

$$T_j \geq T_i + s_i + t_{ij} - M_{ij}(1 - x_{ij}) \quad (i,j) \in A \quad (15)$$

目的関数 (4) は、デポへの帰着を含む総走行時間で、Timefold のソフトスコアと同じである。制約 (5)–(6) は各訪問先の入次数と出次数を 1 にし、すべての訪問先を割り当てる。制約 (7)–(8) により、各車両はデポを高々 1 回出発し、出発したら同じ数だけデポへ戻る。

制約 (9)–(12) は車両の識別を伝える。 $x_{ij} = 1$ のとき (12) から $a_{ik} \leq a_{jk}$ が全 k について成り立ち、(9) と合わせて $a_{ik} = a_{jk}$ となる。したがって、デポ o_k を出たルート上の訪問先はすべて $a_{.k} = 1$ となり、(11) によりそのルートは自分のデポ o_k に戻る。これにより、容量制約 (13) を車両ごとに書ける。

制約 (14)–(15) は時刻の伝播で、到着が早ければ待つことを許す。最大終了時刻は変数 T_i の上界 $l_i - s_i$ で表す。サービス時間が正なので、(15) は訪問先だけからなる閉路も排除する。

3.5 タイトな big-M

big-M 係数はアークごとに時間枠から決める。

$$M_{ij} = l_i + t_{ij} - e_j, \quad M_{kj} = \tau_k + t_{kj} - e_j. \quad (16)$$

$x_{ij} = 0$ のとき (15) の右辺は $T_i + s_i - l_i + e_j \leq e_j$ となる ($T_i + s_i \leq l_i$ より)。これは T_j の下界 e_j 以下なので、制約は効かない。 M_{kj} も同様で、 $M_{kj} < 0$ の場合も $y_{kj} = 0$ のとき右辺は e_j になり妥当である。一律の大きな定数 M を使うより LP 緩和が強くなる。

3.6 部分巡回除去制約

時刻制約 (15) は整数解の部分巡回を排除するが、LP 緩和では big-M が効かず、分数値の小さな閉路が多数現れる。実際、LP 緩和の最適解では 55 件中 41 件の訪問先について、デポから流入するフローが 1 未満だった。そこで部分巡回除去制約 (SEC)

$$\sum_{(i,j) \in A: i,j \in U} x_{ij} \leq |U| - 1 \quad U \subseteq V, |U| \geq 2 \quad (17)$$

を切除平面として加える。分離問題は次のように解く。すべてのデポを 1 つの仮想デポ D にまとめ、LP 解 (x, y) を容量としたネットワークを作る。各訪問先 t について $D-t$ 最小カットを求め、カット値が 1 未満ならシンク側の集合 U に対する (17) が破られている (入次数制約 (5) の下で、 U への流入が 1 未満であることと (17) の違反は同値)。破られた制約を加えて LP を解き直し、違反がなくなるまで繰り返す。

4 AMPL による実装

AMPL の推奨に従い、モデル (.mod) とデータ処理・求解 (Python, amplpy) を分けた。 .dat ファイルや .run ファイルは使わない。

4.1 モデルファイル

リスト 1 にモデルファイル mdvrptw.mod の全体を示す。SEC は添字集合 CUTS と集合族 CUT_SET で定義し、Python から集合の中身を更新するだけでカットを追加できる。

Listing 1 AMPL モデル (mdvrptw.mod)

```
1  # =====
2  # Multi-Depot VRP with Time Windows (Timefold vehicle-routing quickstart)
3  #
4  # 2-index arc formulation with vehicle-specific depot arcs:
5  #   x[i,j]  visit -> visit arcs
6  #   y[k,j]  depot of vehicle k -> visit j   (first visit of vehicle k)
7  #   z[i,k]  visit i -> depot of vehicle k   (last visit of vehicle k)
8  #   a[i,k]  visit i is served by vehicle k  (carries the vehicle identity
9  #         along a route so that each route returns to its own depot and
10 #         respects that vehicle's capacity)
11 #
12 # Timefold constraints reproduced:
13 #   hard : vehicleCapacity, serviceFinishedAfterMaxEndTime
14 #   soft : minimizeTravelTime (total driving time incl. return to depot)
15 # All visits must be assigned (Timefold list variable without unassigned values).
16 # Times are in seconds measured from problem startDateTime.
17 # =====
18
19 set VISITS;
20 set VEHICLES;
21
22 param demand    {VISITS} >= 0;
23 param service   {VISITS} >= 0;           # service duration
24 param ready     {VISITS} >= 0;           # minStartTime
```

```

25 param due      {i in VISITS} >= ready[i] + service[i]; # maxEndTime
26
27 param capacity {VEHICLES} >= 0;
28 param depart   {VEHICLES} >= 0; # departureTime
29
30 param t_vv {VISITS, VISITS} >= 0; # visit -> visit
31 param t_dv {VEHICLES, VISITS} >= 0; # depot -> visit
32 param t_vd {VISITS, VEHICLES} >= 0; # visit -> depot
33
34 # Arc pre-processing: drop arcs that cannot respect time windows / start times
35 set ARCS = {i in VISITS, j in VISITS:
36             i != j and ready[i] + service[i] + t_vv[i,j] + service[j] <= due[j]};
37 set START = {k in VEHICLES, j in VISITS:
38              depart[k] + t_dv[k,j] + service[j] <= due[j]};
39
40 var x {ARCS} binary;
41 var y {START} binary;
42 var z {VISITS, VEHICLES} binary;
43 var a {VISITS, VEHICLES} binary;
44 var T {i in VISITS} >= ready[i], <= due[i] - service[i]; # service start time
45
46 minimize Total_Driving_Time:
47     sum {(i,j) in ARCS} t_vv[i,j] * x[i,j]
48 + sum {(k,j) in START} t_dv[k,j] * y[k,j]
49 + sum {i in VISITS, k in VEHICLES} t_vd[i,k] * z[i,k];
50
51 # Every visit has exactly one predecessor and one successor
52 subject to In_Degree {j in VISITS}:
53     sum {(i,j) in ARCS} x[i,j] + sum {(k,j) in START} y[k,j] = 1;
54
55 subject to Out_Degree {i in VISITS}:
56     sum {(i,j) in ARCS} x[i,j] + sum {k in VEHICLES} z[i,k] = 1;
57
58 # Each vehicle leaves its depot at most once and comes back to the same depot
59 subject to Leave_Depot {k in VEHICLES}:
60     sum {(k,j) in START} y[k,j] <= 1;
61
62 subject to Return_Depot {k in VEHICLES}:
63     sum {i in VISITS} z[i,k] = sum {(k,j) in START} y[k,j];
64
65 # Vehicle identity propagation
66 subject to Assign {i in VISITS}:
67     sum {k in VEHICLES} a[i,k] = 1;
68
69 subject to Start_Identity {(k,j) in START}:
70     y[k,j] <= a[j,k];
71
72 subject to End_Identity {i in VISITS, k in VEHICLES}:
73     z[i,k] <= a[i,k];
74
75 subject to Arc_Identity {(i,j) in ARCS, k in VEHICLES}:
76     x[i,j] + a[i,k] - a[j,k] <= 1;
77
78 # Hard: vehicle capacity
79 subject to Capacity {k in VEHICLES}:
80     sum {i in VISITS} demand[i] * a[i,k] <= capacity[k];
81
82 # Hard: time propagation (waiting allowed; also eliminates subtours).

```

```

83 # Written with arc-specific tight big-M coefficients derived from the time
84 # windows. The indicator form  $x[i,j] = 1 \implies T[j] \geq T[i] + \dots$  is also
85 # valid, but AMPL MP's reformulation grew the model from 3,255 to 10,020
86 # variables with no gain (same incumbent and a similar bound with Gurobi), so
87 # the linear form is used here.
88 subject to Time_From_Depot {(k,j) in START}:
89     T[j] >= depart[k] + t_dv[k,j] - (depart[k] + t_dv[k,j] - ready[j]) * (1 - y[k,j]);
90
91 subject to Time_Between_Visits {(i,j) in ARCS}:
92     T[j] >= T[i] + service[i] + t_vv[i,j] - (due[i] + t_vv[i,j] - ready[j]) * (1 - x[i,j]);
93
94 # Subtour elimination cuts. Not needed for correctness (time propagation
95 # already forbids cycles) but they strengthen the LP relaxation; the sets are
96 # filled by a min-cut separation loop in solve_mdvrptw.py.
97 set CUTS default {};
98 set CUT_SET {CUTS} within VISITS;
99
100 subject to SEC {c in CUTS}:
101     sum {(i,j) in ARCS: i in CUT_SET[c] and j in CUT_SET[c]} x[i,j] <= card(CUT_SET[c]) - 1;

```

4.2 インジケータ制約との比較

AMPL の MP インタフェースでは、時刻制約を

$$x[i,j] = 1 \implies T[j] \geq T[i] + \text{service}[i] + t_{vv}[i,j];$$

のようなインジケータ（含意）制約で書ける。big-M を自分で決める必要がなく、読みやすい。ただし本モデルでは、MP の自動変換によりソルバーに渡るモデルが 3,255 変数から 10,020 変数（うち整数 2,112）に増えた。Gurobi で 300 秒解いた予備実験（SEC なし、Timefold 解を初期解）では、暫定解はどちらも 82,210 で、下界はインジケータ版 76,476、タイトな big-M 版 76,210 と大差なかった。HiGHS ではモデルが大きくなるだけで利点がないため、本稿では線形の big-M 形を採用した。

4.3 Python (amplpy)

主要部分を示す。全体は付録に載せる。リスト 2 は Timefold と同じ走行時間の計算である。

Listing 2 走行時間の計算

```

38 def java_round(v):
39     return math.floor(v + 0.5)
40
41
42 def driving_seconds(p, q):
43     if p == q:
44         return 0
45     lat1, lon1, lat2, lon2 = map(math.radians, (*p, *q))
46     h = math.sin((lat2 - lat1) / 2) ** 2 + math.cos(lat1) * math.cos(lat2) * math.sin((lon2 - lon1) / 2) ** 2
47     meters = java_round(2 * EARTH_RADIUS_M * math.asin(math.sqrt(h)))
48     return java_round(meters / AVERAGE_SPEED_KMPH * 3.6)

```

リスト 3 はモデルを読み込み、pandas の DataFrame と辞書からデータを渡す部分である。環境

変数 `AMPL_PATH` で AMPL 本体の場所を指定できる. `pip` で入る `ampl_module_base` はデモライセンス (500 変数まで) なので, 本モデルの求解には正規ライセンスの AMPL が必要である.

Listing 3 モデルの構築とデータの受け渡し

```

101 def make_ampl():
102     ampl_path = Path(os.environ.get("AMPL_PATH", DEFAULT_AMPL_PATH))
103     return AMPL(Environment(str(ampl_path))) if ampl_path.exists() else AMPL()
104
105
106 def build_model(P):
107     V, K = P["visits"], P["vehicles"]
108     ampl = make_ampl()
109     ampl.read(str(MOD_PATH))
110
111     ampl.set_data(pd.DataFrame.from_dict(
112         {i: dict(demand=v["demand"], service=v["service"], ready=v["ready"], due=v["due"]) for i,
113           v in V.items()},
114         orient="index", "VISITS")
115     ampl.set_data(pd.DataFrame.from_dict(
116         {k: dict(capacity=v["capacity"], depart=v["depart"]) for k, v in K.items()}, orient="
117         index"), "VEHICLES")
118
119     ampl.param["t_vv"] = {(i, j): driving_seconds(V[i]["loc"], V[j]["loc"]) for i in V for j in V
120     }
121     ampl.param["t_dv"] = {(k, j): driving_seconds(K[k]["loc"], V[j]["loc"]) for k in K for j in V
122     }
123     ampl.param["t_vd"] = {(i, k): driving_seconds(V[i]["loc"], K[k]["loc"]) for i in V for k in K
124     }
125     return ampl

```

リスト 4 は SEC の切除平面ループである. `relax_integrality` で LP 緩和を解き, `NetworkX` の最小カットで違反した SEC を探す.

Listing 4 SEC の分離 (切除平面法)

```

163 def load_cuts(ampl, cuts):
164     ampl.set["CUTS"] = list(range(len(cuts)))
165     for c, members in enumerate(cuts):
166         ampl.set["CUT_SET"][c] = members
167
168
169 def separate_sec(P, max_rounds=50):
170     """Cutting-plane loop on the LP relaxation: add violated subtour elimination
171     constraints found by max-flow/min-cut from a super depot to every visit."""
172     ampl = build_model(P)
173     ampl.option["solver"] = "highs"
174     ampl.option["relax_integrality"] = 1
175     cuts, history = [], []
176     for _ in range(max_rounds):
177         ampl.solve(verbose=False)
178         history.append(ampl.get_objective("Total_Driving_Time").value())
179         G = nx.DiGraph()
180         G.add_nodes_from(["D", *P["visits"]])
181         for (i, j), v in ampl.get_variable("x").get_values().to_dict().items():
182             if v > 1e-6:
183                 G.add_edge(i, j, capacity=v)
184         for (k, j), v in ampl.get_variable("y").get_values().to_dict().items():
185             if v > 1e-6:

```

```

186         G.add_edge("D", j, capacity=G.get_edge_data("D", j, {"capacity": 0})["capacity"]
+ v)
187     known = {frozenset(c) for c in cuts}
188     new = set()
189     for t in P["visits"]:
190         value, (_, sink_side) = nx.minimum_cut(G, "D", t)
191         if value < 1 - 1e-4 and frozenset(sink_side) not in known:
192             new.add(frozenset(sink_side))
193     if not new:
194         break
195     cuts += [sorted(c) for c in new]
196     load_cuts(ampl, cuts)
197     return cuts, history

```

リスト 5 は MIP の求解である．初期解（warm）を変数の初期値として渡し，Gurobi のオプション `mip:start=1` で MIP スタートとして使わせる．AMPL の HiGHS ドライバ（1.11.0）にはこのオプションがない．

Listing 5 MIP の求解

```

200 def solve(P, solver, timelimit, warm=None, fix=None, cuts=()):
201     ampl = build_model(P)
202     load_cuts(ampl, cuts)
203     if warm:
204         set_start(ampl, warm)
205     if fix:
206         set_start(ampl, fix, fix=True)
207     ampl.option["solver"] = solver
208     opts = f"outlev=1 timelimit={timelimit} mip:gap=0 mip:bestbound=1 threads=8"
209     if warm:
210         opts += " mip:start=1" # Gurobi only; the HiGHS driver has no MIP start option
211     ampl.option[f"{solver}_options"] = opts
212     t = time.time()
213     ampl.solve()
214     elapsed = time.time() - t
215     status = ampl.solve_result
216     routes = extract_routes(ampl, P)
217     if status not in ("solved", "limit") or sum(map(len, routes.values())) != len(P["visits"]):
218         status, routes = "no solution", None
219     obj = ampl.get_objective("Total_Driving_Time").value() if routes else float("nan")
220     try:
221         bound = ampl.get_value("Total_Driving_Time.bestbound")
222     except Exception:
223         bound = float("nan")
224     n_bin = ampl.get_value("_nvars")
225     n_con = ampl.get_value("_ncons")
226     return dict(status=status, obj=obj, bound=bound, time=elapsed, routes=routes,
227                 nvars=n_bin, ncons=n_con)

```

5 計算実験

5.1 実験環境

計算機は Intel Xeon W-2140B（3.2 GHz, 8 コア 16 スレッド）, macOS 13 である．AMPL（20221013 版）から Gurobi 13.0.0 と HiGHS 1.11.0 を呼び，スレッド数 8，時間上限 600 秒，相

対ギャップ 0 を目標に設定した。すべての MIP には、事前に求めた SEC 32 本を加えた。比較したのは次の手法である。

- **Timefold** : JSON に保存された解。同じ評価関数で再計算すると、ハード制約違反 0, 総走行時間 82,660 秒で一致した。
- **HiGHS, Gurobi** : 初期解なし。
- **Gurobi (ウォームスタート)** : Timefold の解を MIP スタートとして与える。

モデルの妥当性を確かめるため、まず Timefold の解の変数を固定して AMPL モデルを解いた。実行可能で、目的関数値は 82,660 と一致した。

5.2 結果

表 2 に結果を示す。ギャップは (上界 - 下界)/上界, 「Timefold のギャップ」は $(82,660 - \text{下界})/82,660$ で、その下界から保証される Timefold の解の最適性ギャップの上限である。

表 2 計算結果 (時間上限 600 秒, 単位は秒)

手法	総走行時間	下界	ギャップ	Timefold 比	Timefold のギャップ
Timefold	82,660	–	–	–	–
LP 緩和	–	66,625	–	–	19.4%
LP 緩和 + SEC 32 本	–	74,453	–	–	9.9%
HiGHS	解なし	75,399	–	–	8.8%
Gurobi	82,257	77,004	6.4%	–0.49%	6.8%
Gurobi (ウォームスタート)	82,210	77,652	5.5%	–0.54%	6.1%

SEC の切除平面ループは 5 回の LP 求解 (3.7 秒) で収束し、LP 下界を 66,625 から 74,453 へ 11.7% 引き上げた。Gurobi は初期解なしでも Timefold より良い解 82,257 を見つけた。Timefold の解から始めると、600 秒で 113,710 ノードを探索して 82,210 を得た。HiGHS は 600 秒で実行可能解を一つも見つけられなかった。

5.3 ルートの比較

表 3 に車両ごとの比較を、図 1 にルートを示す。Gurobi (ウォームスタート) の解は Timefold の解とほとんど同じで、違いは次の 3 点だけである。

- 午前枠の訪問先 32 を車両 2 から車両 4 へ移す。
- 午後枠の訪問先 55 を車両 4 から車両 5 へ移す。
- 車両 5 の最初の 2 件 (訪問先 10 と 39) の順序を入れ替える。

車両 1, 3, 6 のルートは変わらない。車両 2 は 1,216 秒、車両 4 は 37 秒短くなり、車両 5 は 803 秒長くなって、差し引き 450 秒の改善である。Timefold の局所探索は、2 台の車両の間で訪問先を同時に動かすこの種の改善を見落としていた。

一方、初期解なしの Gurobi の解 (82,257) では、車両 4 の訪問先が 5 件に減り、車両 2 が 7 件を担当するなど、Timefold の解とは異なる割り当てになっていた。総走行時間がほぼ同じでも構造

の違う解が複数あることを示している。

表3 車両ごとの比較 (Timefold / Gurobi ウォームスタート)

車両	容量	訪問数	需要	走行時間 [s]	帰着時刻
1	24	11 / 11	16 / 16	15,859 / 15,859	18:15 / 18:15
2	28	6 / 5	9 / 8	10,835 / 9,619	14:42 / 14:42
3	27	14 / 14	21 / 21	18,016 / 18,016	17:21 / 17:21
4	15	7 / 7	11 / 11	11,762 / 11,725	15:42 / 14:34
5	27	10 / 11	15 / 16	14,849 / 15,652	16:01 / 16:57
6	28	7 / 7	9 / 9	11,339 / 11,339	15:13 / 15:13
計	149	55 / 55	81 / 81	82,660 / 82,210	

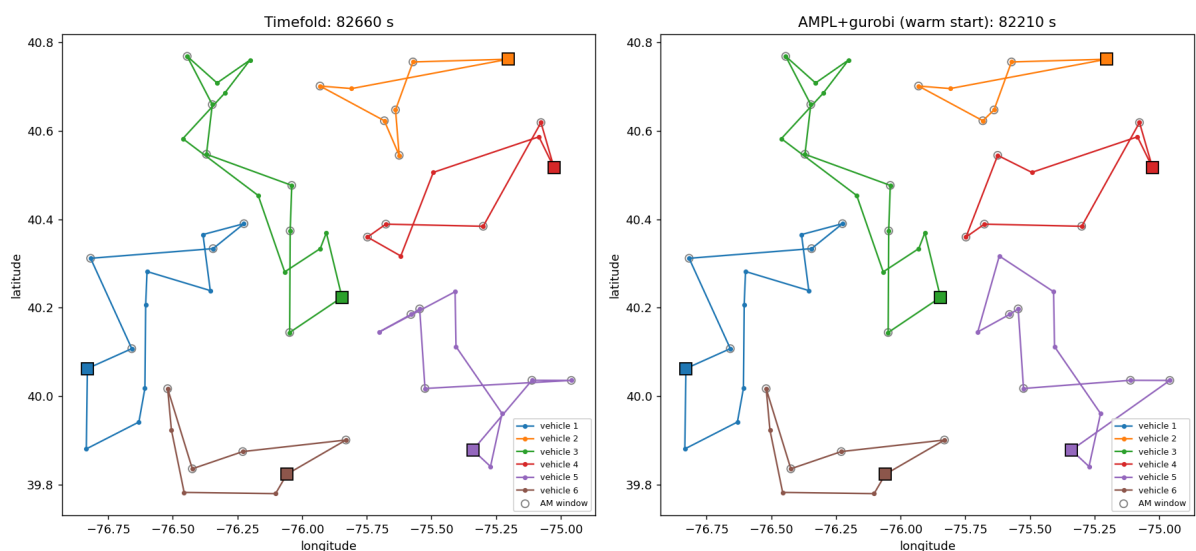


図1 Timefold の解 (左) と Gurobi (ウォームスタート) の解 (右). 四角はデポ, 灰色の円は午前枠の訪問先を表す.

5.4 考察

最終的なギャップ 5.5% の大部分は下界側にあると考えられる. 上界は異なる 3 つの方法 (Timefold, Gurobi の初期解なし・あり) で 82,210–82,660 の狭い範囲に収まった. 一方, 下界は 600 秒の分枝限定法でも 77,652 までしか上がらなかった. 主な原因は時刻制約 (15) の big-M だと考えられる. 時間枠の幅 (4–5 時間) が 1 区間の走行時間 (数十分) に比べて広いので, M_{ij} をタイトにしても LP 緩和では時刻制約がほとんど効かない.

車両識別の弱さも疑い, 車両添字付きアーク x_{ijk} を用いた 3 添字定式化の LP 下界も調べた. SEC を加えた後の LP 下界は 74,517 で, 2 添字版の 74,453 とほとんど変わらなかった. したがって下界の弱さは, マルチデポの扱いではなく時間枠の扱いに起因する. 最適性を証明するには, 時間枠を集合分割 (列生成) の価格付け問題で扱う分枝価格法や, 時間枠に関する妥当不等式の追加が有

望である。

6 まとめ

Timefold の vehicle-routing クイックスタートと同じ複数デポ時間枠付き配送計画問題を MIP として定式化し、AMPL と amply で実装した。走行時間の計算を Timefold と完全に一致させたので、両者の解を同じ尺度で比較できる。55 訪問先・6 台のインスタンスでは、Gurobi が Timefold の解を 0.54% 改善した。得られた下界から、Timefold の解は最適値から高々 6.1% 以内であることを保証した。メタヒューリスティクスの解を MIP の初期解として与えると、改善と品質保証を同時に行えることが確認できた。

付録 A Python コード全体

Listing 6 solve_mdvrtw.py

```
1  """Multi-depot VRPTW (Timefold vehicle-routing quickstart) solved with AMPL/amplpy.
2
3  Reads フィラデルフィア.json (a Timefold quickstart problem + its solution),
4  solves the same problem as a MIP (mdvrptw.mod) and compares the routes.
5
6  Usage:
7      python timefold/vrp2/solve_mdvrtw.py [--timelimit 600] [--solvers highs,gurobi]
8
9  Set AMPL_PATH to an AMPL installation with a full license (the pip
10  ampl_module_base ships a size-limited demo license that cannot hold this model).
11  """
12
13  import argparse
14  import json
15  import math
16  import os
17  import time
18  from datetime import datetime, timedelta
19  from pathlib import Path
20
21  import networkx as nx
22  import pandas as pd
23  from amply import AMPL, Environment
24
25  HERE = Path(__file__).resolve().parent
26  JSON_PATH = HERE.parent / "vrp" / "フィラデルフィア.json"
27  MOD_PATH = HERE / "mdvrptw.mod"
28  OUT_DIR = HERE / "results"
29  DEFAULT_AMPL_PATH = Path.home() / "Documents/ampl/ampl.macos64"
30
31  EARTH_RADIUS_M = 6371000.0
32  AVERAGE_SPEED_KMPH = 50.0
33
34
35  # -----
36  # Data (driving time identical to Timefold's HaversineDrivingTimeCalculator)
37  # -----
38  def java_round(v):
```

```

39     return math.floor(v + 0.5)
40
41
42 def driving_seconds(p, q):
43     if p == q:
44         return 0
45     lat1, lon1, lat2, lon2 = map(math.radians, (*p, *q))
46     h = math.sin((lat2 - lat1) / 2) ** 2 + math.cos(lat1) * math.cos(lat2) * math.sin((lon2 -
47         lon1) / 2) ** 2
48     meters = java_round(2 * EARTH_RADIUS_M * math.asin(math.sqrt(h)))
49     return java_round(meters / AVERAGE_SPEED_KMPH * 3.6)
50
51 def load_problem(path=JSON_PATH):
52     raw = json.loads(path.read_text(encoding="utf-8"))
53     prob = raw["problem"]
54     t0 = datetime.fromisoformat(prob["startDateTime"])
55     sec = lambda s: (datetime.fromisoformat(s) - t0).total_seconds()
56
57     visits = {
58         v["id"]: dict(name=v["name"], loc=v["location"], demand=v["demand"],
59             service=v["serviceDuration"], ready=sec(v["minStartTime"]), due=sec(v["
60             maxEndTime"]))
61         for v in prob["visits"]
62     }
63     vehicles = {
64         k["id"]: dict(loc=k["homeLocation"], capacity=k["capacity"], depart=sec(k["departureTime"]
65         ))
66         for k in prob["vehicles"]
67     }
68     routes = {k["id"]: list(k["visits"]) for k in prob["vehicles"]} # Timefold solution
69     return dict(raw=raw, t0=t0, visits=visits, vehicles=vehicles, timefold_routes=routes,
70         timefold_score=raw["summary"])
71
72 # -----
73 # Route evaluation (Timefold score semantics)
74 # -----
75 def evaluate(P, routes):
76     V, K = P["visits"], P["vehicles"]
77     rows, hard, total = [], 0, 0
78     for k, route in routes.items():
79         t, loc, drive, load, late = K[k]["depart"], K[k]["loc"], 0, 0, 0
80         for i in route:
81             d = driving_seconds(loc, V[i]["loc"])
82             drive += d
83             t = max(t + d, V[i]["ready"]) + V[i]["service"]
84             late += max(0, t - V[i]["due"])
85             load += V[i]["demand"]
86             loc = V[i]["loc"]
87         drive += driving_seconds(loc, K[k]["loc"])
88         back = t + driving_seconds(loc, K[k]["loc"]) if route else K[k]["depart"]
89         over = max(0, load - K[k]["capacity"])
90         hard -= over + late / 60 # Timefold: capacity overflow + late minutes
91         total += drive
92         rows.append(dict(vehicle=k, visits=len(route), demand=load, capacity=K[k]["capacity"],
93             driving_s=drive, return_time=str(P["t0"] + timedelta(seconds=back
94             ))[11:19],

```

```

93         late_s=late, over_capacity=over))
94     assigned = sum(len(r) for r in routes.values())
95     return dict(driving=total, hard=hard, assigned=assigned, per_vehicle=pd.DataFrame(rows))
96
97
98     # -----
99     # AMPL model
100    # -----
101    def make_ampl():
102        ampl_path = Path(os.environ.get("AMPL_PATH", DEFAULT_AMPL_PATH))
103        return AMPL(Environment(str(ampl_path))) if ampl_path.exists() else AMPL()
104
105
106    def build_model(P):
107        V, K = P["visits"], P["vehicles"]
108        ampl = make_ampl()
109        ampl.read(str(MOD_PATH))
110
111        ampl.set_data(pd.DataFrame.from_dict(
112            {i: dict(demand=v["demand"], service=v["service"], ready=v["ready"], due=v["due"]) for i,
113              v in V.items()}),
114            orient="index", "VISITS")
115        ampl.set_data(pd.DataFrame.from_dict(
116            {k: dict(capacity=v["capacity"], depart=v["depart"]) for k, v in K.items()}), orient="
117            index", "VEHICLES")
118
119        ampl.param["t_vv"] = {(i, j): driving_seconds(V[i]["loc"], V[j]["loc"]) for i in V for j in V
120        }
121        ampl.param["t_dv"] = {(k, j): driving_seconds(K[k]["loc"], V[j]["loc"]) for k in K for j in V
122        }
123        ampl.param["t_vd"] = {(i, k): driving_seconds(V[i]["loc"], K[k]["loc"]) for i in V for k in K
124        }
125        return ampl
126
127
128    def set_start(ampl, routes, fix=False):
129        """Load a set of routes as initial values (MIP start) or fix them."""
130        arcs = {tuple(a) for a in ampl.get_set("ARCS").get_values().to_list()}
131        starts = {tuple(a) for a in ampl.get_set("START").get_values().to_list()}
132        x = {a: 0 for a in arcs}
133        y = {a: 0 for a in starts}
134        z, a_ = {}, {}
135        visits = ampl.get_set("VISITS").get_values().to_list()
136        for i in visits:
137            for k in routes:
138                z[i, k] = a_[i, k] = 0
139        for k, r in routes.items():
140            if not r:
141                continue
142            y[k, r[0]] = 1
143            z[r[-1], k] = 1
144            for i in r:
145                a_[i, k] = 1
146            for i, j in zip(r, r[1:]):
147                x[i, j] = 1
148        for name, vals in (("x", x), ("y", y), ("z", z), ("a", a_)):
149            var = ampl.get_variable(name)
150            var.set_values(vals)

```

```

146         if fix:
147             for idx, v in vals.items():
148                 var[idx].fix(v)
149
150
151 def extract_routes(ampl, P):
152     x = {k: v for k, v in ampl.get_variable("x").get_values().to_dict().items() if v > 0.5}
153     y = {k: v for k, v in ampl.get_variable("y").get_values().to_dict().items() if v > 0.5}
154     succ = {i: j for (i, j) in x}
155     routes = {k: [] for k in P["vehicles"]}
156     for (k, j) in y:
157         while j is not None:
158             routes[k].append(j)
159             j = succ.get(j)
160     return routes
161
162
163 def load_cuts(ampl, cuts):
164     ampl.set["CUTS"] = list(range(len(cuts)))
165     for c, members in enumerate(cuts):
166         ampl.set["CUT_SET"][c] = members
167
168
169 def separate_sec(P, max_rounds=50):
170     """Cutting-plane loop on the LP relaxation: add violated subtour elimination
171     constraints found by max-flow/min-cut from a super depot to every visit."""
172     ampl = build_model(P)
173     ampl.option["solver"] = "highs"
174     ampl.option["relax_integrality"] = 1
175     cuts, history = [], []
176     for _ in range(max_rounds):
177         ampl.solve(verbose=False)
178         history.append(ampl.get_objective("Total_Driving_Time").value())
179         G = nx.DiGraph()
180         G.add_nodes_from(["D", *P["visits"]])
181         for (i, j), v in ampl.get_variable("x").get_values().to_dict().items():
182             if v > 1e-6:
183                 G.add_edge(i, j, capacity=v)
184         for (k, j), v in ampl.get_variable("y").get_values().to_dict().items():
185             if v > 1e-6:
186                 G.add_edge("D", j, capacity=G.get_edge_data("D", j, {"capacity": 0})["capacity"]
187 + v)
188         known = {frozenset(c) for c in cuts}
189         new = set()
190         for t in P["visits"]:
191             value, (_, sink_side) = nx.minimum_cut(G, "D", t)
192             if value < 1 - 1e-4 and frozenset(sink_side) not in known:
193                 new.add(frozenset(sink_side))
194         if not new:
195             break
196         cuts += [sorted(c) for c in new]
197         load_cuts(ampl, cuts)
198     return cuts, history
199
200 def solve(P, solver, timelimit, warm=None, fix=None, cuts=()):
201     ampl = build_model(P)
202     load_cuts(ampl, cuts)

```

```

203     if warm:
204         set_start(ampl, warm)
205     if fix:
206         set_start(ampl, fix, fix=True)
207     ampl.option["solver"] = solver
208     opts = f"outlev=1 timelimit={timelimit} mip:gap=0 mip:bestbound=1 threads=8"
209     if warm:
210         opts += " mip:start=1" # Gurobi only; the HiGHS driver has no MIP start option
211     ampl.option[f"{solver}_options"] = opts
212     t = time.time()
213     ampl.solve()
214     elapsed = time.time() - t
215     status = ampl.solve_result
216     routes = extract_routes(ampl, P)
217     if status not in ("solved", "limit") or sum(map(len, routes.values())) != len(P["visits"]):
218         status, routes = "no solution", None
219     obj = ampl.get_objective("Total_Driving_Time").value() if routes else float("nan")
220     try:
221         bound = ampl.get_value("Total_Driving_Time.bestbound")
222     except Exception:
223         bound = float("nan")
224     n_bin = ampl.get_value("_nvars")
225     n_con = ampl.get_value("_ncons")
226     return dict(status=status, obj=obj, bound=bound, time=elapsed, routes=routes,
227                nvars=n_bin, ncons=n_con)
228
229
230 # -----
231 # Output helpers
232 # -----
233 def to_timefold_json(P, routes, name):
234     raw = json.loads(json.dumps(P["raw"]))
235     raw["name"] = name
236     for veh in raw["problem"]["vehicles"]:
237         veh["visits"] = routes[veh["id"]]
238     ev = evaluate(P, routes)
239     raw["summary"] = f"訪問先 {ev['assigned']} 件・車両 {len(routes)} 台・総走行時間 {ev['driving']} 秒 (AMPL)"
240     return raw
241
242
243 def plot_routes(P, results, path):
244     import matplotlib.pyplot as plt
245
246     V, K = P["visits"], P["vehicles"]
247     cmap = plt.get_cmap("tab10")
248     fig, axes = plt.subplots(1, len(results), figsize=(7 * len(results), 6.5), squeeze=False)
249     for ax, (label, routes) in zip(axes[0], results.items()):
250         for c, (k, r) in enumerate(routes.items()):
251             pts = [K[k]["loc"]] + [V[i]["loc"] for i in r] + [K[k]["loc"]]
252             ax.plot([p[1] for p in pts], [p[0] for p in pts], "-o", ms=3, lw=1.3, color=cmap(c),
253                    label=f"vehicle {k}")
254             ax.plot(K[k]["loc"][1], K[k]["loc"][0], "s", ms=10, color=cmap(c), mec="black")
255             am = [v["loc"] for v in V.values() if v["ready"] < 5 * 3600]
256             ax.scatter([p[1] for p in am], [p[0] for p in am], s=40, facecolors="none", edgecolors="gray",
257                       label="AM window")
258             ax.set_title(f"{label}: {evaluate(P, routes)['driving']} s")

```

```

258     ax.set_xlabel("longitude")
259     ax.set_ylabel("latitude")
260     ax.legend(fontsize=7, loc="lower right")
261 fig.tight_layout()
262 fig.savefig(path, dpi=130)
263
264
265 # -----
266 # Experiment
267 # -----
268 def main():
269     ap = argparse.ArgumentParser()
270     ap.add_argument("--timelimit", type=int, default=600)
271     ap.add_argument("--solvers", default="highs,gurobi")
272     args = ap.parse_args()
273     OUT_DIR.mkdir(exist_ok=True)
274
275     P = load_problem()
276     tf = P["timefold_routes"]
277     ev_tf = evaluate(P, tf)
278     print(f"Timefold JSON: {P['timefold_score']}")
279     print(f" re-evaluated: hard={ev_tf['hard']}, driving={ev_tf['driving']} s")
280     print(ev_tf["per_vehicle"].to_string(index=False))
281
282     # Sanity check: the Timefold routes must be feasible in the AMPL model with the same
283     # objective
284     chk = solve(P, "highs", 60, fix=tf)
285     print(f"\nTimefold routes fixed in AMPL model: {chk['status']}, obj={chk['obj']:.0f}")
286     assert abs(chk["obj"] - ev_tf["driving"]) < 0.5
287
288     t = time.time()
289     cuts, lp_hist = separate_sec(P)
290     print(f"\nLP relaxation: {lp_hist[0]:.1f} -> {lp_hist[-1]:.1f} with {len(cuts)} subtour cuts
291         "
292         f"({len(lp_hist)} rounds, {time.time() - t:.1f}s)")
293
294     rows = [dict(method="Timefold (JSON)", status="-", driving_s=ev_tf["driving"], bound_s=None,
295                 gap_pct=None, time_s=None, hard=ev_tf["hard"], assigned=ev_tf["assigned"]),
296             dict(method="LP relaxation", status="-", driving_s=None, bound_s=lp_hist[0]),
297             dict(method=f"LP relaxation + {len(cuts)} SECs", status="-", driving_s=None, bound_s=
298                 lp_hist[-1])]
299     route_sets = {"Timefold": tf}
300     solvers = args.solvers.split(",")
301     runs = [(s, None) for s in solvers] + [("gurobi", tf)] * ("gurobi" in solvers)
302     for solver, warm in runs:
303         label = f"AMPL+{solver}" + (" (warm start)" if warm else "")
304         print(f"\n=== {label}, timelimit={args.timelimit}s ===")
305         r = solve(P, solver, args.timelimit, warm=warm, cuts=cuts)
306         if r["routes"] is None:
307             print(f"{label}: no feasible solution, bound={r['bound']:.1f} time={r['time']:.1f}s")
308             rows.append(dict(method=label, status=r["status"], driving_s=None, bound_s=r["bound"]
309                             ],
310                             gap_pct=None, time_s=r["time"], hard=None, assigned=None))
311             continue
312         ev = evaluate(P, r["routes"])
313         assert abs(ev["driving"] - r["obj"]) < 0.5 and ev["hard"] == 0 and ev["assigned"] == len(P
314             ["visits"])
315         gap = 100 * (r["obj"] - r["bound"]) / r["obj"] if r["bound"] == r["bound"] else None

```

```

311     rows.append(dict(method=label, status=r["status"], driving_s=ev["driving"], bound_s=r["
bound"],
312                     gap_pct=gap, time_s=r["time"], hard=ev["hard"], assigned=ev["assigned"
]))
313     route_sets[label] = r["routes"]
314     print(f"{label}: {r['status']} obj={r['obj']:.0f} bound={r['bound']:.1f} time={r['time
']:.1f}s "
315           f"(vars={r['nvars']:.0f}, cons={r['ncons']:.0f})")
316
317 table = pd.DataFrame(rows)
318 table["vs_timefold_pct"] = 100 * (table["driving_s"] - ev_tf["driving"]) / ev_tf["driving"]
319 table["timefold_gap_pct"] = 100 * (ev_tf["driving"] - table["bound_s"]) / ev_tf["driving"]
320 table.to_csv(OUT_DIR / "comparison.csv", index=False)
321 print("\n" + table.to_string(index=False))
322
323 best_label = min((l for l in route_sets if l != "Timefold"),
324                 key=lambda l: evaluate(P, route_sets[l])["driving"])
325 best = route_sets[best_label]
326 for k in P["vehicles"]:
327     print(f"vehicle {k}\n Timefold: {tf[k]}\n {best_label}: {best[k]}")
328 evaluate(P, best)["per_vehicle"].to_csv(OUT_DIR / "best_per_vehicle.csv", index=False)
329 (OUT_DIR / "routes.json").write_text(json.dumps(route_sets, ensure_ascii=False, indent=1),
encoding="utf-8")
330 (OUT_DIR / "フィラデルフィア_ampl.json").write_text(
331     json.dumps(to_timefold_json(P, best, "フィラデルフィア (AMPL)"), ensure_ascii=False, indent
=1), encoding="utf-8")
332 plot_routes(P, {"Timefold": tf, best_label: best}, OUT_DIR / "routes_comparison.png")
333 print(f"\nResults written to {OUT_DIR}")
334
335
336 if __name__ == "__main__":
337     main()

```